

Internal documentation The notes a programmer leaves inside the code — for the next person, who is often future-you.

Kai, a Year 12 student, built CourtBook : a small app that lets students book the school gym's basketball courts at lunch. It worked. Then Kai graduated and left.

What this key knowledge covers

This summary distils everything you need for **KK10 — Internal documentation The notes a programmer leaves inside the code — for the next person, who is often future-you.** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Meet CourtBook — and the bug Mia inherited

Kai, a Year 12 student, built **CourtBook**: a small app that lets students book the school gym's basketball courts at lunch. It worked. Then Kai graduated and left.

Three weeks later, courts are being double-booked. The job of fixing it lands on **Mia**, a Year 11 who has *never seen this code before*. She opens the file. Whether she fixes the bug in ten minutes or loses a whole afternoon depends almost entirely on one thing: did Kai leave internal documentation?

That last clause matters. A comment changes *nothing* about what the program outputs. It exists purely to make the code understandable to a human reader — which is exactly why a beginner is tempted to skip it, and exactly why examiners ask about it.

Internal vs external documentation

Don't mix these up — it's a classic trap.

The pieces of internal documentation

Internal documentation comes in two broad forms: **comments** (written notes the language ignores) and **formatting** (how the code is laid out). Comments themselves come in a few

recognisable flavours. Click any line below to see what it's doing.

The header / metadata block

A block of comments at the *top* of a file or module that records who, what, when and why. The first thing Mia reads. Click each line:

Warning comments

A comment that flags something *surprising* or fragile — a place where the obvious change would break things. These earn their keep.

The three purposes of internal documentation

If you remember nothing else, remember these three. They are the answer to almost every "explain the purpose(s)" question on this dot point.

Describe what the data structures and code structures do, and *why* they were chosen.

A comment notes that bookings are kept in a **list of records** (not a single array) because each booking carries several fields — court, slot, student — of different types.

Help a future programmer (or future-you) debug, update and extend the solution quickly and safely.

Mia reads the comment on `is_clash()` and instantly sees the double-booking bug: it only checked the court, not the time. The comment told her the *intended* rule, so the gap was obvious.

Mark and explain unfinished sections — placeholders left for later — so they aren't mistaken for working code.

A **stub** for the future "email confirmation" feature is a comment-only placeholder: `# STUB: send_email()` — returns True for now. It documents intent without pretending to work.

Explain vs justify — the distinction that splits the band

Purpose 1 contains two verbs, and students lose marks by treating them as the same word. They are not.

Quick test: if your comment could start with "*because...*", it's a justification. If it starts with "*this...*" or "*loops/checks/sets...*", it's an explanation. Strong answers in an exam often give one of each.

Layout is documentation too

Comments get the attention, but **how the code is laid out** is internal documentation as well — it shapes how fast a human can read structure.

Indentation, blank lines between logical chunks, and meaningful names all communicate *without a single comment*. Compare these two versions of the same function — identical behaviour, very different readability.

The right-hand version barely needs a comment — the **meaningful names** (`slot_is_free`, `existing_bookings`) and the **indentation** document the logic by themselves. This is why naming conventions (KK09) and internal documentation are siblings: good names are documentation you don't have to write twice.

Signal vs noise — more comments isn't better

A common misconception is "good documentation = lots of comments." Wrong. A comment that just restates the code adds **noise** — it clutters the file and goes stale the moment the code changes. Good comments add **signal**: the *why*, the *gotcha*, the intent the code can't show.

Rule of thumb to repeat in your head: comment the why, not the what. The code already shows *what* it does; your comment should capture what the code *can't* say.

The Maintenance Time Machine

Here's Kai's `is_clash()` module as a 3D object. Flip the **documentation** switch and watch two things happen: the comment-tags fade in around the code, and the meter shows roughly how long it takes a *brand-new* programmer like Mia to understand and safely change it. Drag to rotate.

Time-to-understand simulator

The numbers are illustrative, but the relationship is real and well documented in software engineering: developers spend far more time *reading* existing code than writing new code, so anything that speeds reading pays for itself many times over.

Sort it out

Drag each chip into the correct bin. Each sorter saves your score — get them all to all correct.

P·E·L writing trainer

Short-answer marks come from structure, not length. Use **Point** → **Evidence** → **Link**. Draft an answer to the prompt below; the keyword tags light up as you hit the ideas examiners reward.

Prompt: "Explain one purpose of internal documentation in a software solution." (2 marks)

State the purpose plainly. e.g. "One purpose is to support maintenance."

Show how — name a concrete mechanism (a comment explaining a structure, a header block).

Connect to the benefit — a future programmer can debug/update the code faster and more safely.

Practice questions

Five VCE-style questions. For each, try it yourself first, then reveal a weak and a strong response with annotations and the marking guide. Self-mark honestly — your score tracks at the top.

Six-card recap

Explanatory notes *inside* the source code — comments + layout. For humans; ignored at runtime.

Internal lives in the code. External = manuals, help guides, specs — separate documents.

Explain & justify structures · support maintenance · describe stubs.

Explain = what/how. Justify = why (often starts with "because...").

Indentation, white space and meaningful names are documentation too.

Comment the *why*, not the obvious *what*. More comments $\neq$ better.

"Internal documentation is comments and layout written inside the source code to explain and justify the structures, support maintenance and mark stubs — read by humans and ignored when the program runs."

Common traps & misconceptions

Exam trap

TRAP 01 Calling a user manual "internal documentation" User manuals, help guides and technical specs are **external**. Internal documentation lives *inside the source code*. ✓ Internal = comments + code layout. External = separate documents.

Exam trap

TRAP 02 Thinking comments change how the program runs Comments are **ignored by the compiler/interpreter**. They never affect output, speed or behaviour. They benefit humans, not execution. ✓ "Comments are stripped at compile time / skipped by the interpreter."

Exam trap

TRAP 03 Treating "explain" and "justify" as the same thing Explain = *what/how*. Justify = *why this choice*. A "justify" comment can usually start with "because...". ✓ Strong answers give one explanation *and* one justification.

Exam trap

TRAP 04 "My code works, so comments are a waste" Working code and maintainable code are different goals. The cost of missing documentation appears later — when *someone else* (or future-you) must change it. ✓ Tie the value to **maintenance**, debugging and handover.

Exam trap

TRAP 05 Equating "more comments" with "better" Comments that restate the code are **noise** — they clutter and go stale. Quality means commenting the *why*, not the obvious *what*. ✓ Mention signal vs noise if a question asks about effective documentation.

Exam trap

TRAP 06 Forgetting layout counts Internal documentation isn't only comments. **Indentation, white space and meaningful names** document structure too. ✓ Name layout/formatting as a form of internal documentation.

Exam trap

TRAP 07 Saying documentation is "only for other people" The most common reader is **future-you**, who has forgotten the details. Maintenance includes your own later edits. ✓ "for the next programmer — often the original author months later."

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Define • 2 marks

The term *internal documentation* and give one example.

✗ A weak answer

It's the documents that explain how to use the program, like a manual or instructions. 0/2. This describes *external* documentation. It misses the core idea that internal documentation lives *inside the code*, and gives an example that is actually external.

✓ A strong answer

Internal documentation is explanatory material written inside the source code — such as comments — that helps a programmer understand and maintain the code, and which is ignored when the program runs. Example: a comment explaining why a list was used to store bookings. 2/2. Correct definition (inside the code, for humans, ignored at runtime) + a valid internal example.

How the marks are awarded

- 1 mark — definition: explanatory notes *within the source code*, for human readers, not affecting how it runs.
- 1 mark — a valid internal example (comment, header block, meaningful naming or layout).
- No mark if the example is external (manual, help guide, spec).

Q2. Explain · 4 marks

two purposes of internal documentation in a software solution.

✗ A weak answer

So the code is documented and looks professional, and so there are lots of comments to help people. Comments make the program better and easier. 1/4. Vague and circular ("documented... so it's documented"). No two *distinct* purposes, no mechanism, no clear benefit. "Looks professional" isn't a purpose.

✓ A strong answer

One purpose is to **explain and justify the structures used**: a comment can state that bookings are stored in a list of records and why that structure was chosen, so the teammate understands the design quickly. A second purpose is to **support maintenance**: clear comments and a header block let the teammate debug and extend the code without having to reverse-engineer it, reducing errors and saving time. 4/4. Two distinct purposes, each with a mechanism (comment on structure / header block) *and* a benefit (understanding, faster safer maintenance).

How the marks are awarded

- 2 marks per purpose (×2): 1 for naming a valid purpose, 1 for explaining how it helps / its benefit.
- Valid purposes: explain & justify structures · support maintenance/debugging · describe stubs/unfinished sections.
- The two purposes must be *distinct* — restating one in different words caps the answer at 2.

Q3. Distinguish · 2 marks

Between a comment that *explains* code and a comment that *justifies* a design decision. Give an example of each.

✗ A weak answer

Explaining tells you about the code and justifying also tells you about the code. For example "# this loops" and "# this loops through the list". 0/2. Both examples are explanations — no justification shown. The distinction (what/how vs why) is never actually made.

✓ A strong answer

An **explaining** comment describes *what* the code does or *how* it works, e.g. # loops through each booking to find a clash. A **justifying** comment gives the *reason a choice was made*, e.g. # a list is used because the daily booking count is not known in advance. 2/2. Clear what/how vs why contrast, with one valid example of each.

How the marks are awarded

- 1 mark — states the distinction: explain = what/how the code works; justify = why a decision was made.
- 1 mark — a correct example of each type.

Q4. Describe · 2 marks

A programmer writes `# STUB: send_email()` — returns True for now. Describe what a *stub* is and why documenting it is useful.

✗ A weak answer

A stub is a small bit of code. It's good to write a comment so the code is documented. 0/2.

Doesn't say a stub is a *placeholder for unwritten code*, and the "why" just repeats the question without a real benefit.

✓ A strong answer

A stub is a placeholder for a part of the program not yet written — here, the email feature is faked to return True. Documenting it makes clear the section is **incomplete**, so another programmer doesn't mistake it for finished, working code and knows it still needs building. 2/2. Correct definition of a stub + a genuine benefit (prevents confusion, flags unfinished work for maintenance).

How the marks are awarded

- 1 mark — stub = placeholder for an unwritten/incomplete part, often returning a fixed value.
- 1 mark — why documenting helps: signals the code is unfinished so it isn't mistaken for working code / guides future development.